LAPORAN TUGAS INDIVIDU PEMBANGUNAN RESTful API CRUD SEDERHANA MENGGUNAKAN NODE.JS DAN EXPRESS

IDENTITAS MAHASISWA

Nama : HAIQAL ALY HAZAMY
Perguruan Tinggi: STMIK Antar Bangsa
Mata Kuliah : Pemrograman Web Service / API
Tahun Akademik : 2026/2027 Ganjil Modul / Pertemuan: Pertemuan 8 - Tugas Mandiri Sub-CPMK-1.1

I. PENDAHULUAN & ARSITEKTUR SISTEM

Laporan ini disusun untuk memenuhi tugas pembuatan web service RESTful API sederhana menggunakan Node.js dan framework Express.js. Aplikasi ini bertugas mengelola data "Biodata" secara CRUD (Create, Read, Update, Delete).

Spesifikasi Data & Penyimpanan:

- Penyimpanan Data: Array JavaScript (In-Memory Data Store)
- Atribut Data :
    - id (Number, Unique, Auto-Increment)
    - nama (String)
    - umur (Number)
    - alamat (String)
    - pekerjaan (String)

Format Respon Standar (JSON): Semua endpoint menghasilkan balasan berformat JSON konsisten: { "status": "success" | "fail", "message": "Pesan informasi operasi", "data": { ... } | [...] | null }

II. SPESIFIKASI ENDPOINT & KODE PROGRAM

Kode program dibangun dalam dua file utama:

1. package.json

JSON

```
None
{
  "name": "tugas-api-biodata",
  "version": "1.0.0",
  "description": "Tugas Pembangunan RESTful API CRUD Sederhana",
  "main": "server.js",
  "scripts": {
    "start": "node server.js"
  },
  "dependencies": {
    "express": "^4.18.2"
  }
}
```

2. server.js

JavaScript

```
None
const express = require('express');
const app = express();
const PORT = 9000;

app.use(express.json());

let biodatas = [];
let nextId = 1;

// 1. GET /api/biodata - Menampilkan seluruh data
app.get('/api/biodata', (req, res) => {
    res.status(200).json({
        status: 'success',
        message: 'Berhasil mengambil seluruh data biodata',
        data: biodatas
    });
```

```
});

// 2. GET /api/biodata/:id - Detail 1 data
app.get('/api/biodata/:id', (req, res) => {
    const id = parseInt(req.params.id);
    const biodata = biodatas.find(b => b.id === id);

    if (!biodata) {
        return res.status(404).json({
            status: 'fail',
            message: `Biodata dengan ID ${id} tidak ditemukan`,
            data: null
        });
    }

    res.status(200).json({
        status: 'success',
        message: 'Berhasil mengambil detail biodata',
        data: biodata
    });
});

// 3. POST /api/biodata - Menambah data baru
app.post('/api/biodata', (req, res) => {
    const { nama, umur, alamat, pekerjaan } = req.body;

    if (!nama || !umur || !alamat || !pekerjaan) {
        return res.status(400).json({
            status: 'fail',
            message: 'Semua field (nama, umur, alamat, pekerjaan)
wajib diisi',
            data: null
        });
    }

    const newBiodata = {
        id: nextId++,
        nama,
        umur,
```

```
        alamat,
        pekerjaan
    };

    biodatas.push(newBiodata);

    res.status(201).json({
        status: 'success',
        message: 'Berhasil menambahkan biodata baru',
        data: newBiodata
    });
});

// 4. PUT /api/biodata/:id - Memperbarui data
app.put('/api/biodata/:id', (req, res) => {
    const id = parseInt(req.params.id);
    const index = biodatas.findIndex(b => b.id === id);

    if (index === -1) {
        return res.status(404).json({
            status: 'fail',
            message: `Biodata dengan ID ${id} tidak ditemukan`,
            data: null
        });
    }

    const { nama, umur, alamat, pekerjaan } = req.body;

    biodatas[index] = {
        ...biodatas[index],
        nama: nama || biodatas[index].nama,
        umur: umur || biodatas[index].umur,
        alamat: alamat || biodatas[index].alamat,
        pekerjaan: pekerjaan || biodatas[index].pekerjaan
    };

    res.status(200).json({
        status: 'success',
        message: 'Berhasil memperbarui data biodata',
```

```
        data: biodatas[index]
    });
});

// 5. DELETE /api/biodata/:id - Menghapus data
app.delete('/api/biodata/:id', (req, res) => {
    const id = parseInt(req.params.id);
    const index = biodatas.findIndex(b => b.id === id);

    if (index === -1) {
        return res.status(404).json({
            status: 'fail',
            message: `Biodata dengan ID ${id} tidak ditemukan`,
            data: null
        });
    }

    const deletedData = biodatas.splice(index, 1);

    res.status(200).json({
        status: 'success',
        message: 'Berhasil menghapus biodata',
        data: deletedData[0]
    });
});

app.listen(PORT, () => {
    console.log(`Server API berjalan di
http://localhost:${PORT}`);
});
```

III. HASIL PENGUJIAN ENDPOINT (DOCUMENTASI cURL & RESPON)

1. Pengujian POST /api/biodata (Tambah Data Berhasil - HTTP 201 Created)

Perintah cURL: curl -X POST http://localhost:9000/api/biodata

-H "Content-Type: application/json"

-d '{ "nama": "Haiqal Aly Hazamy", "umur": 20, "alamat": "Jl. Rahasia 1", "pekerjaan": "Mahasiswa" }'

Respon Server: { "status": "success", "message": "Berhasil menambahkan biodata baru", "data": { "id": 1, "nama": "Haiqal Aly Hazamy", "umur": 20, "alamat": "Jl. Kemerdekaan No. 1", "pekerjaan": "Mahasiswa" } }

2. Pengujian POST /api/biodata (Validasi Input Gagal - HTTP 400 Bad Request)

Perintah cURL: curl -X POST http://localhost:9000/api/biodata

-H "Content-Type: application/json"

-d '{ "nama": "Haiqal Aly Hazamy" }'

Respon Server: { "status": "fail", "message": "Semua field (nama, umur, alamat, pekerjaan) wajib diisi", "data": null }

3. Pengujian GET /api/biodata (Ambil Seluruh Data - HTTP 200 OK)

Perintah cURL: curl -X GET http://localhost:9000/api/biodata

Respon Server: { "status": "success", "message": "Berhasil mengambil seluruh data biodata", "data": [{ "id": 1, "nama": "Haiqal Aly Hazamy", "umur": 20, "alamat": "Jl. Rahasia 1", "pekerjaan": "Mahasiswa" }] }

4. Pengujian GET /api/biodata/1 (Detail Data Berhasil - HTTP 200 OK)

Perintah cURL: curl -X GET http://localhost:9000/api/biodata/1

Respon Server: { "status": "success", "message": "Berhasil mengambil detail biodata", "data": { "id": 1, "nama": "Haiqal Aly Hazamy", "umur": 20, "alamat": "Jl. Rahasia 1", "pekerjaan": "Mahasiswa" } }

5. Pengujian GET /api/biodata/999 (ID Tidak Ada - HTTP 404 Not Found)

Perintah cURL: curl -X GET http://localhost:9000/api/biodata/999

Respon Server: { "status": "fail", "message": "Biodata dengan ID 999 tidak ditemukan", "data": null }

6. Pengujian PUT /api/biodata/1 (Update Data - HTTP 200 OK)

Perintah cURL: curl -X PUT http://localhost:9000/api/biodata/1

-H "Content-Type: application/json"

-d '{ "umur": 21, "pekerjaan": "Software Engineer" }'

Respon Server: { "status": "success", "message": "Berhasil memperbarui data biodata", "data": { "id": 1, "nama": "Haiqal Aly Hazamy", "umur": 21, "alamat": "Jl. Rahasia 1", "pekerjaan": "Software Engineer" } }

7. Pengujian DELETE /api/biodata/1 (Hapus Data - HTTP 200 OK)

Perintah cURL: curl -X DELETE http://localhost:9000/api/biodata/1

Respon Server: { "status": "success", "message": "Berhasil menghapus biodata", "data": { "id": 1, "nama": "Haiqal Aly Hazamy", "umur": 21, "alamat": "Jl. Rahasia 1", "pekerjaan": "Software Engineer" } }

IV. KESIMPULAN

Berdasarkan hasil pengujian di atas, seluruh spesifikasi tugas RESTful API telah terpenuhi dengan baik:

1. Fungsi CRUD (Create, Read, Update, Delete) berjalan normal.

2. Penanganan array JavaScript menggunakan method find, findIndex, push, dan splice.

3. Handling error untuk data tidak ditemukan (404) dan input tidak lengkap (400) telah berfungsi sesuai kaidah REST API.

4. Format respon JSON konsisten dan status code HTTP tepat (200, 201, 400, 404).